ASSIGNMENT 3 – CO3

Q1 – Handling Failure in a Bank ATM Simulation

Programming in Java | 6 Marks | Bloom's Level: K4 – Analyse

1. Aim and problem interpretation

Implement an ATM withdrawal simulation that validates account and amount input, checks the balance, verifies access to the card-reader file, handles failures using try/catch (including multi-catch), and executes finally. The sample account balance is Rs. 10,000.

2. Relevant Throwable hierarchy

```
Object
  ■■■ Throwable
      ■■■ Error
      ■   ■■■ VirtualMachineError (e.g. OutOfMemoryError)
      ■■■ Exception
          ■■■ IOException (checked)
          ■   ■■■ FileNotFoundException
          ■■■ RuntimeException (unchecked)
              ■■■ InputMismatchException
              ■■■ IllegalArgumentException
              ■■■ IllegalStateException
```

Error represents a serious JVM or environment problem and is generally not handled as an ordinary application failure. IOException is checked and must be caught or declared. RuntimeException subclasses are unchecked and need not be declared.

3. Failure classification

Failure point	Exception/category	Checked?	Where handled
Card-reader file missing/inaccessible	FileNotFoundException / IOException	Yes	catch (IOException e)
Letters for account or amount	InputMismatchException	No	Multi-catch
Non-positive account number	IllegalArgumentException	No	Multi-catch
Zero/negative withdrawal	IllegalArgumentException	No	Multi-catch
Amount exceeds balance	IllegalStateException	No	catch (IllegalStateException e)
Severe JVM resource failure	OutOfMemoryError example	Error	Not caught by application

4. Catch order and finally

The IOException catch handles the specific checked file failure. The multi-catch handles InputMismatchException and IllegalArgumentException, which are unrelated types. IllegalStateException has its own specific catch. Specific catches must precede broader catches. If catch (Exception e) is placed first, later catches for its subclasses become unreachable and the compiler reports an error. The finally block runs when control exits the try/catch structure in normal and handled-exception paths here; it prints the session-ending message and closes the Scanner. It is not an absolute guarantee if the JVM or process is forcibly terminated.

5. Test plan and catch/finally tracing

Test	Input/setup	Expected catch path	Finally
Valid	123456; 2500	No catch; success	Yes
Letters in account	abc	InputMismatchException multi-catch	Yes
Letters in amount	123456; xyz	InputMismatchException multi-catch	Yes
Negative amount	123456; -500	IllegalArgumentException multi-catch	Yes
Insufficient balance	123456; 12000	IllegalStateException catch	Yes

Test	Input/setup	Expected catch path	Finally
Missing file	Rename card-reader.txt	IOException catch	Yes

These are designed test paths. Execute each case and record the exact observed output; add terminal screenshots to the repository.

6. Uncaught exception and stack trace

Run `java ATMWithdrawal --uncaught`. This deliberately throws ArithmeticException outside a catch block. A typical trace has this form:

```
Exception in thread "main" java.lang.ArithmeticException: Demonstration of an uncaught exception
    at ATMWithdrawal.demonstrateUncaughtException(ATMWithdrawal.java:...)
    at ATMWithdrawal.main(ATMWithdrawal.java:...)
```

The first line identifies the thread, exception class and message. The first stack frame identifies the method and source line where it was thrown. The next frame identifies its caller, main. Exact line numbers depend on the source file.

7. Broader catch placed before a narrower catch

```
try { /* operation */ }
catch (Exception e) { /* broad handler */ }
catch (IOException e) { /* compile-time error: unreachable */ }
```

IOException is a subclass of Exception. The first catch already handles every IOException, so the later IOException catch can never execute. The compiler rejects it. Put specific catches first.

8. Joint design question: should one catch (Exception e) be used?

Not as the default ATM design. A single broad catch hides the distinction between invalid input, insufficient funds and card-reader failure, making recovery and user messages less precise. It may be acceptable at a final application boundary for logging an unexpected exception and preventing a crash, after expected failures have been handled specifically. It does not catch Error, because Error is not a subclass of Exception.

9. Alternative: validation and error codes

An exception-free version checks tokens before conversion using Scanner.hasNextInt() and hasNextDouble(). Invalid tokens are rejected or consumed and the method returns a status such as -1; success returns 0. It still checks account positivity, amount positivity, available balance and file-access status. This makes expected input failures explicit but requires repeated checks and status-code handling by callers. Exception-based code can keep the normal path clearer, while creating and throwing exceptions is relatively expensive.

10. Time and space complexity

Operation	Time	Additional space
Validation and balance check	O(1) per transaction	O(1)
Error-code return	O(1) after checks	O(1)
Create/throw exception	Stack-trace capture depends on call depth d; approximately O(d)	O(d) trace data

The number of input fields and checks is fixed, so validation is O(1) time and O(1) auxiliary space per transaction. Exception construction and stack-trace capture add overhead dependent on call-stack depth. Returning an error code avoids the stack-trace cost. Actual runtime should be measured on the execution environment.

11. Conclusion

The ATM simulation demonstrates classification and handling of checked and unchecked failures using specific catch blocks, multi-catch and finally-based cleanup. The error-code alternative is suitable for expected validation failures, while exceptions provide structured propagation for exceptional conditions.

No student identity details are included in this report. Add actual execution screenshots and measured observations to your submission.